

12_RMarkdown

Bailey Dixon

2026-04-26

Contents

R Markdown	2
Including Plots	2
1.1 How I Created a New R Markdown File	3
1.2 Install and Load Packages	3
Section 2: Formatting Text Using RMarkdown Syntax	4
2.1. Headings	4
2.2 Italics	6
2.3 Bold	6
2.4 Bold and Italics	7
2.5 Bulleted list	7
2.6 Numbered List	8
Section 3: Inserting Code Chunks and Running Basic R Code	9
Part A: Create a Scatterplot	17
Part B: Create a Boxplot	18
Part C: Create a Summary Table	19
Part D: Compare the Visuals	20
Part E: Now you try	20
	21
Section 5: Improve and Finalize Your R Markdown Report	21
Step 1: Understanding Inline R Code	21
Step 2: Improving Readability by Hiding Code	22
Step 3: Removing Warnings and Messages	23
Step 4: Hiding Code AND Output with <code>include=FALSE</code>	25
Step 5: Controlling Figure Size	26

Step 6: Adding a Table of Contents	27
Step 7: Adding Hyperlinks	28
Step 8: Writing a Strong Conclusion	28
Step 9: Final Check and Knit	29

R Markdown

This is an R Markdown document. Markdown is a simple formatting syntax for authoring HTML, PDF, and MS Word documents. For more details on using R Markdown see <http://rmarkdown.rstudio.com>.

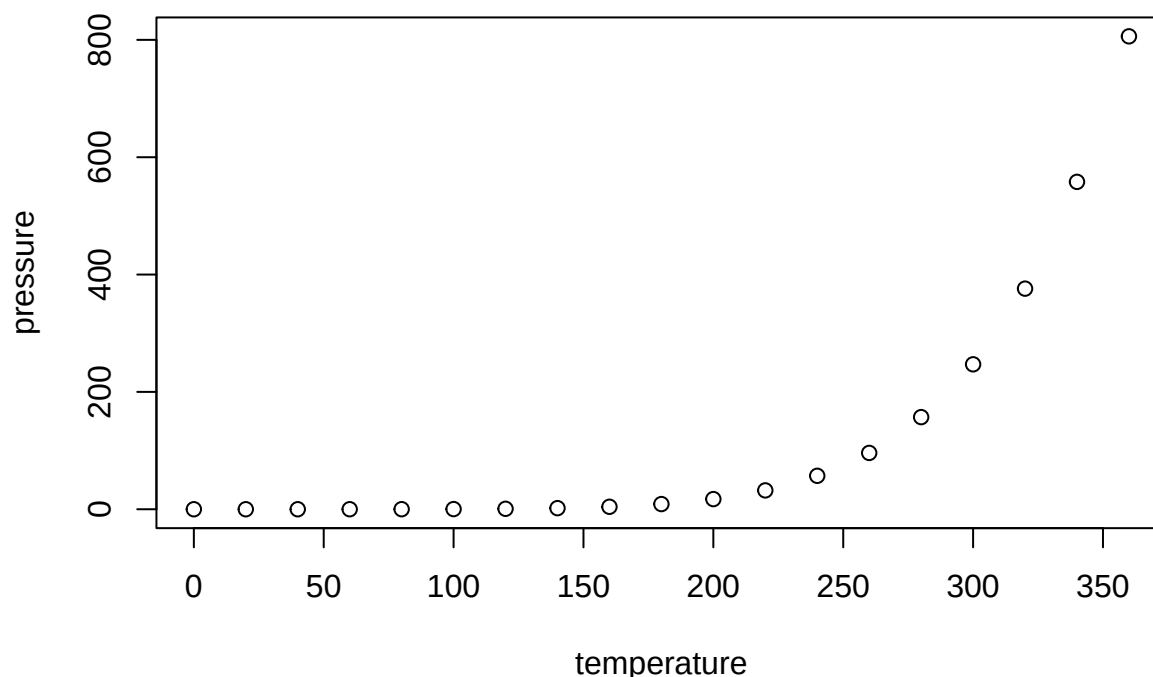
When you click the **Knit** button a document will be generated that includes both content as well as the output of any embedded R code chunks within the document. You can embed an R code chunk like this:

```
summary(cars)
```

```
##      speed      dist
##  Min.   : 4.0    Min.   :  2.00
## 1st Qu.:12.0    1st Qu.: 26.00
##  Median :15.0    Median : 36.00
##   Mean  :15.4    Mean   : 42.98
## 3rd Qu.:19.0    3rd Qu.: 56.00
##   Max.  :25.0    Max.    :120.00
```

Including Plots

You can also embed plots, for example:



Note that the `echo = FALSE` parameter was added to the code chunk to prevent printing of the R code that generated the plot.

[R Markdown documentation] (<https://rmarkdown.rstudio.com>)

1.1 How I Created a New R Markdown File

.....

To begin, open RStudio and create a new R Markdown file.

Go to **File > New File > R Markdown**.

QUESTION: What do you think happens when the new file is opened?

ANSWER: A new .rmd file is created that I will insert my own info into

.....

1.2 Install and Load Packages

.....

Before knitting your file, make sure the required packages are installed and loaded.

TASK: Run the following in the Console:

```
install.packages("rmarkdown") install.packages("knitr") library(rmarkdown) library(knitr) install.packages("tinytex")
tinytex::install_tinytex()
```

QUESTION: Why are these packages needed? ANSWER: in order to build and save rmarkdown files and use the knit function

..... ## 1.3 Modify the YAML Header

At the top of the file, you will see the YAML header between two sets of — This section controls the document settings.

TASK: Change the YAML header so that it includes: a new title your name as the author the date PDF output

Example: title: “My First R Markdown File” author: “Jaslyn Davenport” date: “April 2026” output: pdf_document

QUESTION: What is the purpose of the YAML header? ANSWER: it gives the document a title, the authors can enter their names, and determine the output type

..... ##1.4 Save and Knit file

After editing the file, save it as an .Rmd file. Then click the Knit button at the top of RStudio.

If you get an error message about LaTeX or TinyTeX, install TinyTeX by running: `install.packages(“tinytex”) tinytex::install_tinytex()` Then click Knit again.

QUESTION: What happens when you click Knit? ANSWER: it pulls open the pdf file I am currently working on

QUESTION: What type of file is created? ANSWER: PDF

..... ##1.5 Final observation After knitting, compare the .Rmd file to the final PDF.

QUESTION: How is the output file different from the source file? ANSWER: the output file is just the text from the rmd file (which is written within the rstudio console) in PDF format

Section 2: Formatting Text Using RMarkdown Syntax

Author: Arnav Pandey (a_pandey2@uncg.edu)

Objective: learn how to format the appearance of your document in your chosen output format.

.....

2.1. Headings

.....

TASK: Read the question below, click **Knit** to render the document and return here to answer the question.

QUESTION: What difference do you notice about the above line that starts with **# Section 2: Formatting a Report Using RMarkdown Syntax** compared to the line **Objective: learn to format how your document appears in the output of your choice?**

ANSWER: ‘Section 2’ text is slightly larger, bolded and separated by solid lines, while the ‘objective’ line is smaller, not bold, and followed by a dotted line

Placing a hashtag (#) at the beginning of a line turns it into a heading. RMarkdown supports six levels of headings.

TASK: Click Knit to observe how different heading levels below appear in the rendered output.

```
# Line 1: This is an example to show different levels of headings in RMarkdown.  
## Line 2: Line 1: This is an example to show different levels of headings in RMarkdown.  
### Line 3: This is an example to show different levels of headings in RMarkdown.  
#### Line 4: This is an example to show different levels of headings in RMarkdown.  
##### Line 5: This is an example to show different levels of headings in RMarkdown.  
##### Line 6: This is an example to show different levels of headings in RMarkdown.
```

QUESTION: What happens to the size of text when you increase the number of hashtags (#)?

ANSWER: it gets smaller, but the bolding remains the same!

In the example above, the higher the number of hashtags, the lower the heading level (smaller font).

QUESTION: How many hashtags would you use if you wanted highest level heading (largest font)?

ANSWER: 1 hashtag

QUESTION: In the rendered output, do Line 5 and Line 6 from above example appear in the same line?

ANSWER: yes, which could be useful for writing a caption for a figure

TASK: Go to the metadata for this assignment and change the output from `pdf_document` to `html_document`.

QUESTION: Did your Line 5 and Line 6 appear in different lines?

ANSWER: yes

Sometimes the way the program renders output differs between formats. You can leave two spaces at the end of the line for a line break.

TASK: Change the output back to `pdf_document`.

TASK: **Go back** to all the above example lines “Line 1” thru “Line 6” within single set of three backticks “`. Go back above and do as below:

```
# Line 1: This is an example to show different levels of headings in RMarkdown.  
## Line 2: Line 1: This is an example to show different levels of headings in RMarkdown.  
### Line 3: This is an example to show different levels of headings in RMarkdown.  
#### Line 4: This is an example to show different levels of headings in RMarkdown.  
##### Line 5: This is an example to show different levels of headings in RMarkdown.  
##### Line 6: This is an example to show different levels of headings in RMarkdown.
```

.....

2.2 Italics

.....

You may sometimes want to italicize text. RMarkdown allows you to do this easily. To make your text italics, you can enclose it within single asterisk `*[text]*`.

Line 1: This is an example to show how to make text italics.

TASK: Rewrite Line 1 below and only italicize the word “italics.” Leave two spaces at the end of sentence for a line break.

ANSWER: Line 1: This is an example to show how to make text *italics*.

You may be wondering if there is another way to make the text italics, and in fact, there is. You can also enclose your text within single underscore `_[text]_`.

TASK: In the paragraph below, make “Arnav,” “Biological Data Wrangling and Visualization” and “RMarkdown” italics in the following paragraph using both the methods (asterisks and underscore) mentioned above at least once.

My name is *Arnav* Pandey. I am currently enrolled in *Biological Data Wrangling and Visualization* (BIO 457/657). This is a TASK: to use *RMarkdown* to make a text output italics.

.....

2.3 Bold

.....

Sometimes you may want to bold your text to emphasize its importance. RMarkdown supports that as well.

TASK: Read the question below, click Knit, and return to answer the question below.

QUESTION: Which of the following lines gives you bold text?

Line 1. This is an example to show how to make text bold.

Line 2: This is an example to show how to make text bold.

ANSWER: line 2

You can use two asterisks before and after the text to make it bold (`**[text]**`).

There is also another way to make a text bold by enclosing your text within two underscores(`__[text]__`).

Don't believe me? Knit to see if the line below is bold.

Line 3: This is an example to show how to make text bold.

Like you practiced with italics, you can make any word or phrase within a sentence bold. See example of it below when you hit Knit.

Line 5: This is an example to show how to make text **bold**.
Line 4: This is an **example** to show how to make text bold.

TASK: In the paragraph below, make “Arnav,” “Biological Data Wrangling and Visualization” and “RMarkdown” bold in the following paragraph using both the methods (asterisks and underscores) mentioned above at least once.

My name is **Arnav** Pandey. I am currently enrolled in **Biological Data Wrangling and Visualization** (BIO 457/657). This is a TASK: to use **RMarkdown** to make a text output bold.

.....

2.4 Bold and Italics

.....

Is there a way to make a text both bold and italics at the same time? Indeed, there is! You can enclose your text within three asterisks (******[text]******). or underscores (***___[text]___***).

TASK: In the paragraph below, make “Arnav,” “Biological Data Wrangling and Visualization” and “RMarkdown” bold italics using both the methods mentioned above at least once.

My name is ***Arnav*** Pandey. I am currently enrolled in ***Biological Data Wrangling and Visualization*** (BIO 457/657). This is a TASK: to use ***RMarkdown*** to make a text output bold.

In sections 2.2, 2.3, and 2.4, you learnt how to make text italics, bold, or bold+italics by enclosing the text within one, two, and three asterisks or underscores, respectively. You might be tempted to start enclosing a text by underscore and end it using asterisk. For example:

```
*italics_  
__bold**  
***bold italics____
```

Above lines will not give your desired output, so you will have to be consistent with how you enclose your texts. Use the same symbols at both ends!

.....

2.5 Bulleted list

.....

In RMarkdown, you can list your items in the RMarkdown as an unordered (bulleted) list.

To do that, you can follow these steps. Step 1: leave an empty line before list. Step 2: Use ‘+’, ‘-’, or ‘*’ followed by single space before you write your list item.

Step 3: To add another item, leave two spaces at the end of the previous item and go to a new line below and repeat Step 2.

TASK: Knit to see how the items below appear in a list in the output.

- Apple
- Banana
- Raspberry

You can also nest subitems within an item. If you want a nested list as such, just indent the subitem that you want within an item with two spaces. Below is an example of it.

TASK: Hit Knit to look at the output of the lines below:

- Apple
- Banana
- Berries
 - Blueberries
 - Strawberries
 - Raspberries
 - Blackberries
 - Cranberries
- Papaya

TASK: Copy the above list below, add an item “Melon” and nest different types of melons as its subitems. Hit Knit to see your output.

- Apple
- Banana
- Berries
 - Blueberries
 - Strawberries
 - Raspberries
 - Blackberries
 - Cranberries
- Papaya
- Melon
 - Watermelon
 - Cantaloupe
 - Honeydew

.....

2.6 Numbered List

.....

Getting a numbered list is even easier compared to unordered or bulleted list. To generate a numbered list, you can follow these steps. Step 1: Leave an empty line before list. Step 2: Use uppercase/lowercase English alphabets, Arabic numerals, uppercase or lowercase roman numerals followed by space before you write your list item. Be consistent how you want to list. Step 3: To add another item, leave two spaces at the end of the previous item and go to a new line below and repeat Step 2.

- I. Apple
- II. Banana
- III. Raspberry

You can also nest subitems within an item. If you want a nested list as such, just indent the subitem that you want within an item with three spaces. You can use different order format for your subitems than your primary item as well. Below is an example of it. **Please be careful with spaces!**

- 1. Apple
- 2. Banana
- 3. Berries
 - 1. Blueberries
 - 2. Strawberries
 - 3. Raspberries
 - 4. Blackberries
 - 5. Cranberries
- 4. Papaya

Note: If you want to use Roman Numerals, you may have to leave an empty line between items on list.

Section 3: Inserting Code Chunks and Running Basic R Code

Author: Michelle Bautista Canuto (m_bautistac@uncg.edu) We can insert a code chunk by clicking on the green-button with a “c” on the inside and a “+” on the upper left. Click “R”. We can insert functions within the chunk of code. We will see this next.

QUESTION: What is a code chunk? (hint look back at the slides in Canvas) ANSWER: within a code chunk, everything is seen as R code rather than just text, used for plotting, assigning variables, etc

TASK: Practice inserting a code chunk. Run a simple calculation such as “2+2” below {r}. Run this code by clicking on the green side-triangle button:

Next, let’s assign variables. Run the following variables:

```
x<-10
y<-5
```

To view the contents of the variable x, just type ‘x’ and run

```
x
```

```
## [1] 10
```

TASK: Write a chunk of code multiplying the variables x and y.

```
x*y
```

```
## [1] 50
```

QUESTION: What was the result? ANSWER: 50

We will analyze the built-in 'mtcars' dataset. Run the following chunk of code.

```
head(mtcars)
```

```
##           mpg  cyl  disp  hp  drat    wt  qsec vs  am  gear  carb
## Mazda RX4      21.0   6  160 110  3.90  2.620 16.46  0   1    4    4
## Mazda RX4 Wag  21.0   6  160 110  3.90  2.875 17.02  0   1    4    4
## Datsun 710     22.8   4  108  93  3.85  2.320 18.61  1   1    4    1
## Hornet 4 Drive  21.4   6  258 110  3.08  3.215 19.44  1   0    3    1
## Hornet Sportabout 18.7   8  360 175  3.15  3.440 17.02  0   0    3    2
## Valiant        18.1   6  225 105  2.76  3.460 20.22  1   0    3    1
```

QUESTION: What do you think the head() function does? ANSWER: shows the first 6 rows of a dataset

To display the summary statistics for the dataset use the summary() function. Run the following chunk of code.

```
summary(mtcars)
```

```
##           mpg           cyl           disp           hp
##  Min.   :10.40  Min.   :4.000  Min.   : 71.1  Min.   : 52.0
## 1st Qu.:15.43  1st Qu.:4.000  1st Qu.:120.8  1st Qu.: 96.5
## Median :19.20  Median :6.000  Median :196.3  Median :123.0
## Mean   :20.09  Mean   :6.188  Mean   :230.7  Mean   :146.7
## 3rd Qu.:22.80  3rd Qu.:8.000  3rd Qu.:326.0  3rd Qu.:180.0
## Max.   :33.90  Max.   :8.000  Max.   :472.0  Max.   :335.0
##           drat           wt           qsec           vs
##  Min.   :2.760  Min.   :1.513  Min.   :14.50  Min.   :0.0000
## 1st Qu.:3.080  1st Qu.:2.581  1st Qu.:16.89  1st Qu.:0.0000
## Median :3.695  Median :3.325  Median :17.71  Median :0.0000
## Mean   :3.597  Mean   :3.217  Mean   :17.85  Mean   :0.4375
## 3rd Qu.:3.920  3rd Qu.:3.610  3rd Qu.:18.90  3rd Qu.:1.0000
## Max.   :4.930  Max.   :5.424  Max.   :22.90  Max.   :1.0000
##           am           gear           carb
##  Min.   :0.0000  Min.   :3.000  Min.   :1.000
## 1st Qu.:0.0000  1st Qu.:3.000  1st Qu.:2.000
## Median :0.0000  Median :4.000  Median :2.000
## Mean   :0.4062  Mean   :3.688  Mean   :2.812
## 3rd Qu.:1.0000  3rd Qu.:4.000  3rd Qu.:4.000
## Max.   :1.0000  Max.   :5.000  Max.   :8.000
```

Within a new chunk of code, we will code to import the entire mtcars dataset. Run the following code by clicking on the green side-triangle button for "Run Current Chunk"

```
data(mtcars)
```

QUESTION: After running the previous code, view the dataframe of mtcars. How many variables are in the mtcars dataset? ANSWER: 11

Or you can view the dataset in the console by typing “mtcars” and click ‘Run Selected Lines’. mtcars

To find the number of rows in the dataset, we can use the nrow() function within a new chunk of code. Run the following code.

```
nrow(mtcars)
```

```
## [1] 32
```

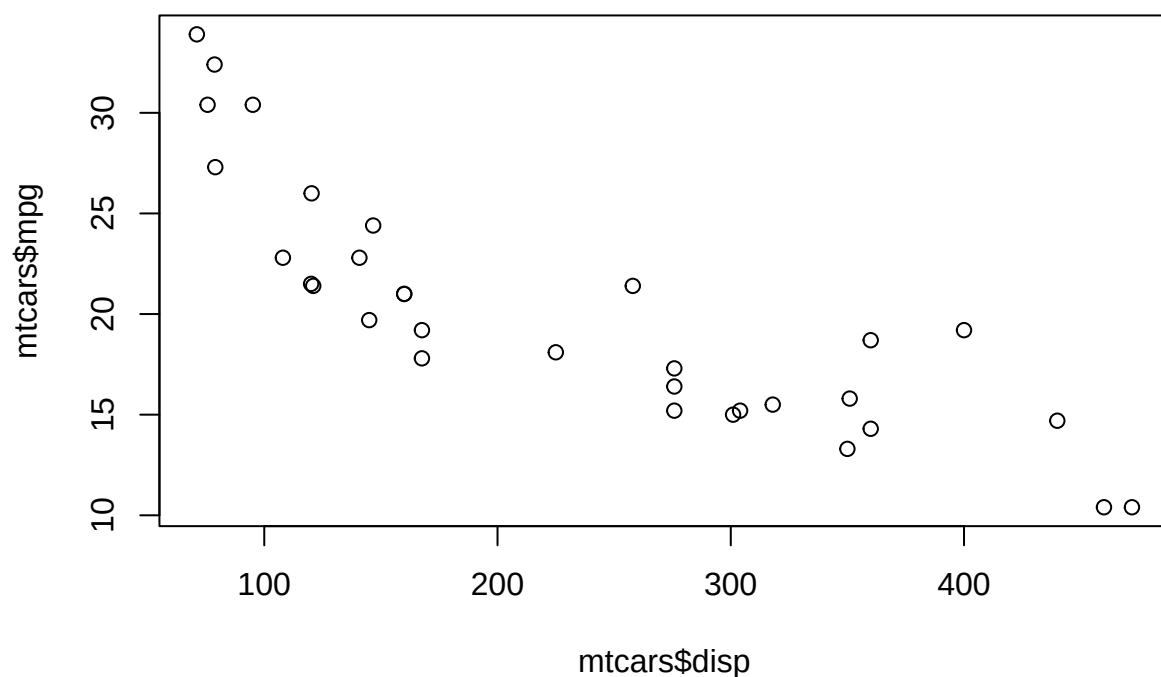
TASK: Write a code to find the number of columns in the ‘mtcars’ dataset using the ncol() function. Remember to use a new chunk of code.

```
ncol(mtcars)
```

```
## [1] 11
```

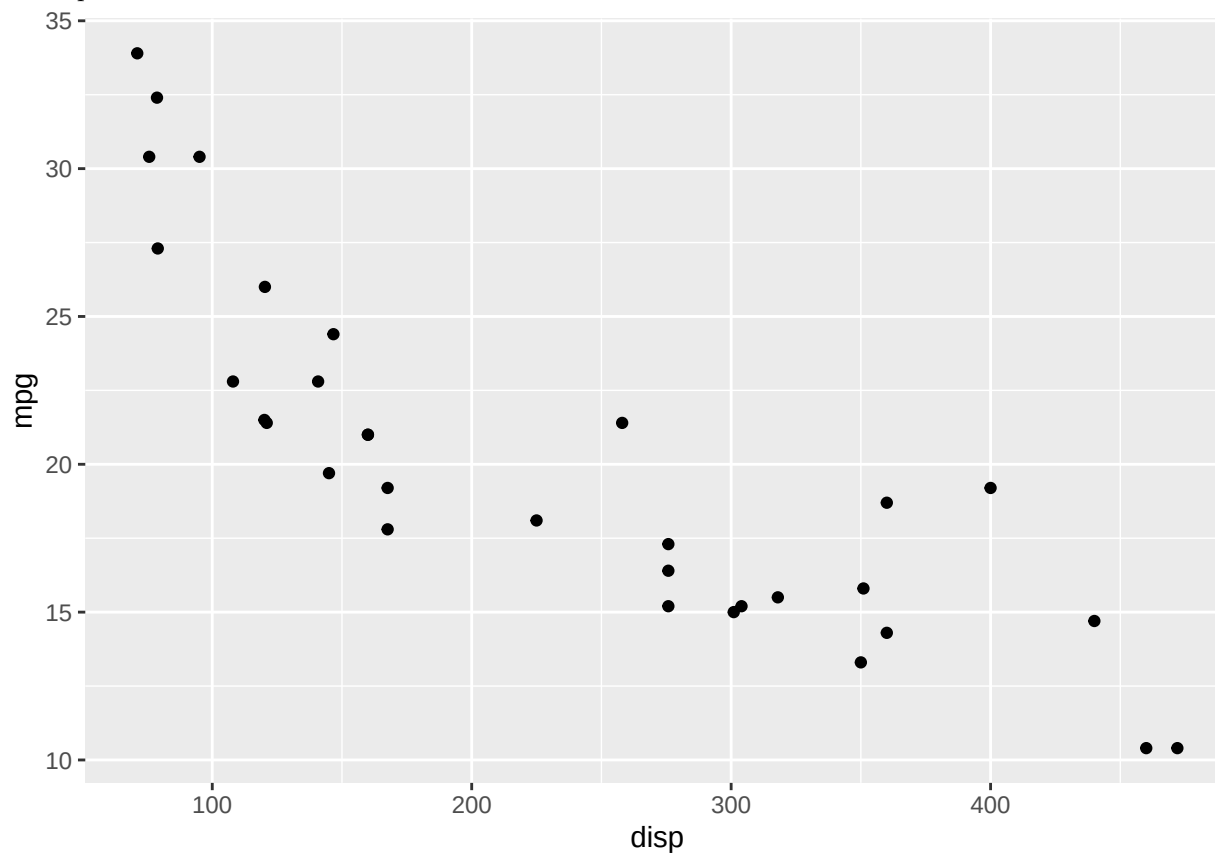
QUESTION: What is the result showing in your script? ANSWER: 11

The ‘disp’ column in the dataset stands for engine displacement. Engine displacement tells you how big the engine is in cubic inches. The ‘mpg’ column stands for fuel efficiency in miles per gallon. Lets plot a graph showing fuel efficiency over displacement for each car. Run the following code.

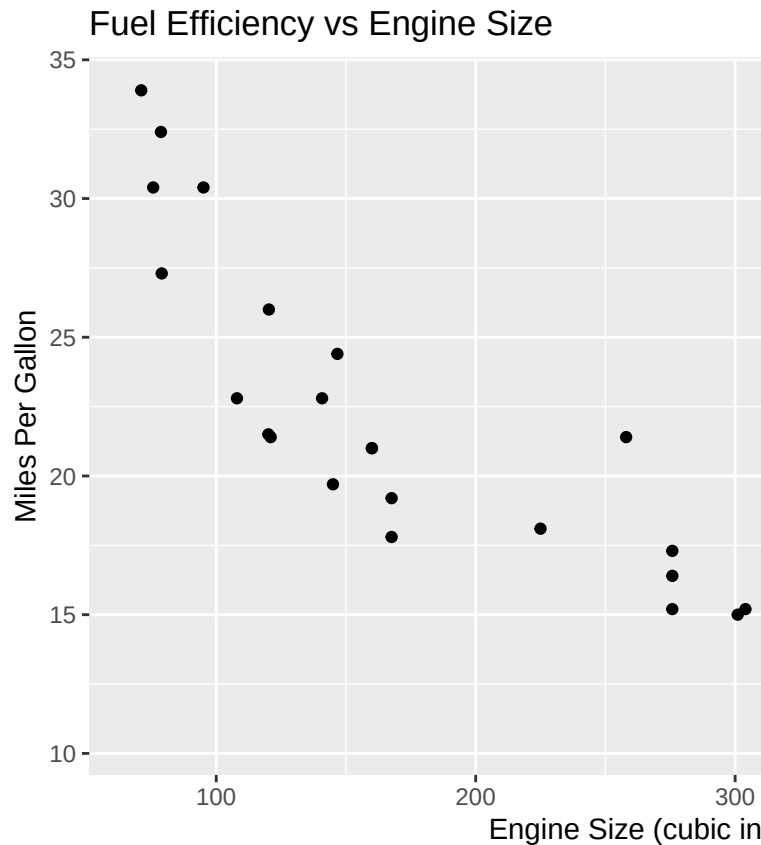


QUESTION: What trend is the plot showing? ANSWER: cars with higher mpg have lower engine size (disp)

We can also use ggplot to run this within our chunk of code. This permits our plot to be displayed in the final report once “knit”.



Let's change the labels of the x-axis to 'Engine Size (cubic inches)' Let's change the y-axis to 'Miles Per Gal-



lon' Let's add a title called "Fuel Efficiency vs Engine Size".

We could add a legend to show which car each point corresponds to. But first, we should add a column displaying the car names using dplyr.

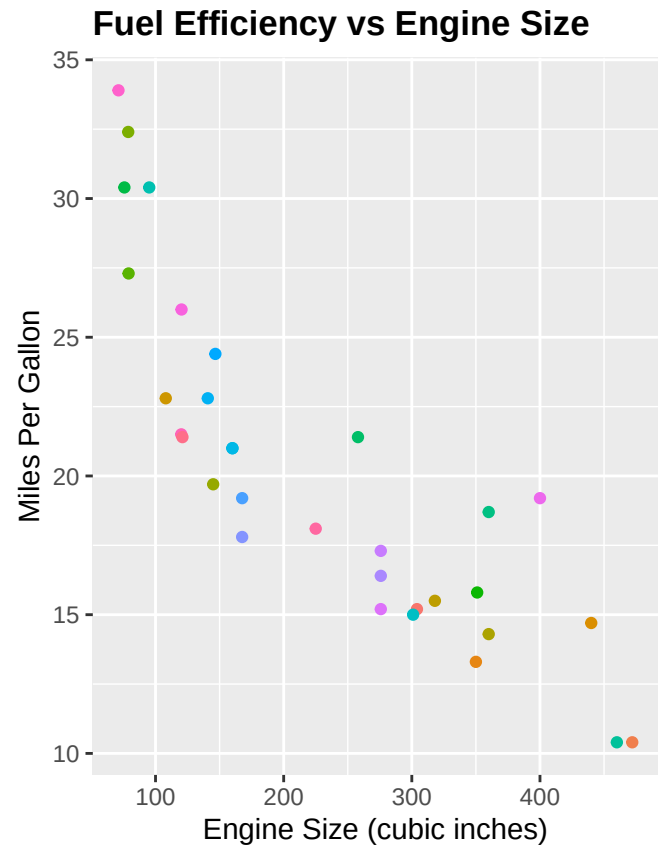
```
library(dplyr)
```

```
##
## Attaching package: 'dplyr'

## The following objects are masked from 'package:stats':
##
##   filter, lag

## The following objects are masked from 'package:base':
##
##   intersect, setdiff, setequal, union
```

```
mtcars<-mtcars %>%
  mutate(car_name=rownames(mtcars))
```



Lets ‘bold’ the title Let’s add a legend to differentiate each point.

Lets organize the fuel efficiency from highest to lowest. Run the following.

```
mtcars %>%
  select(mpg, disp) %>%
  arrange(desc(mpg))
```

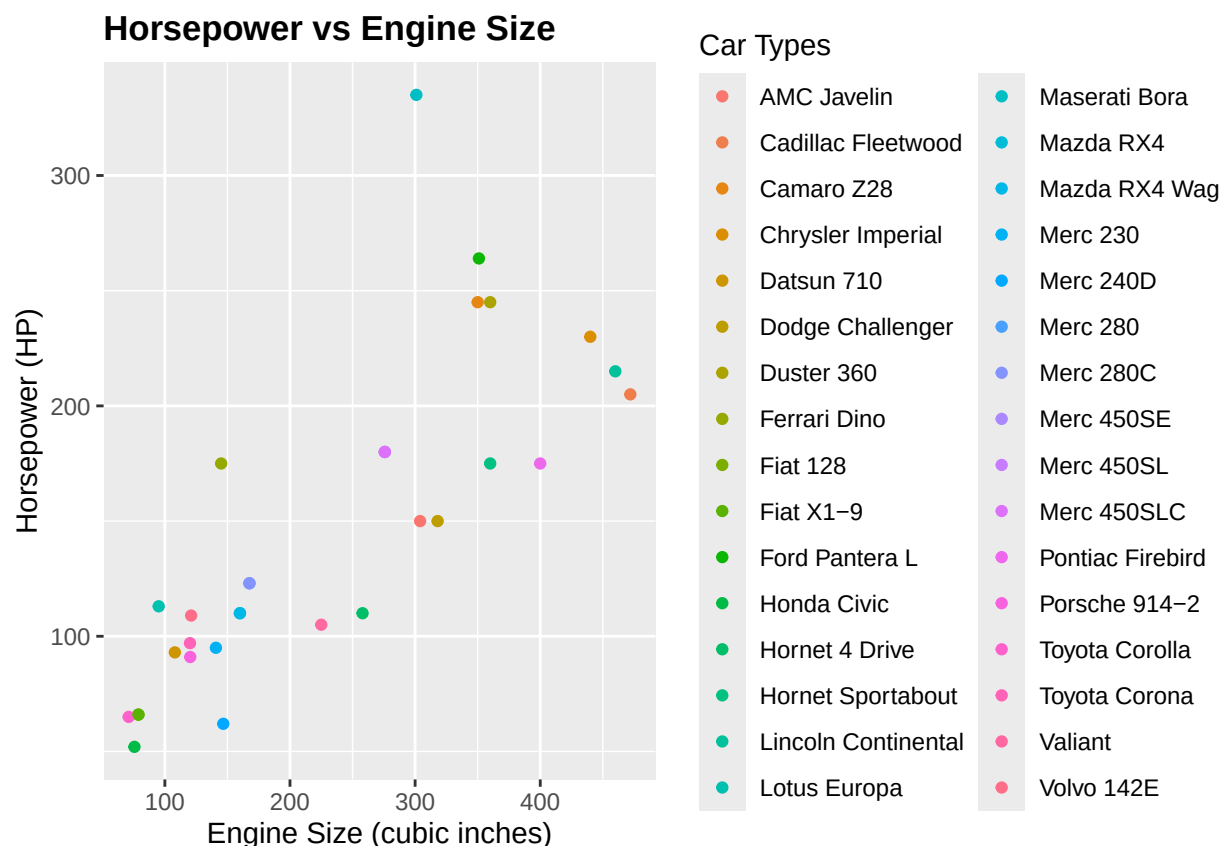
##	mpg	disp
## Toyota Corolla	33.9	71.1
## Fiat 128	32.4	78.7
## Honda Civic	30.4	75.7
## Lotus Europa	30.4	95.1
## Fiat X1-9	27.3	79.0
## Porsche 914-2	26.0	120.3
## Merc 240D	24.4	146.7
## Datsun 710	22.8	108.0
## Merc 230	22.8	140.8
## Toyota Corona	21.5	120.1
## Hornet 4 Drive	21.4	258.0
## Volvo 142E	21.4	121.0
## Mazda RX4	21.0	160.0
## Mazda RX4 Wag	21.0	160.0
## Ferrari Dino	19.7	145.0
## Merc 280	19.2	167.6
## Pontiac Firebird	19.2	400.0
## Hornet Sportabout	18.7	360.0
## Valiant	18.1	225.0

```
## Merc 280C          17.8 167.6
## Merc 450SL        17.3 275.8
## Merc 450SE        16.4 275.8
## Ford Pantera L    15.8 351.0
## Dodge Challenger   15.5 318.0
## Merc 450SLC       15.2 275.8
## AMC Javelin       15.2 304.0
## Maserati Bora     15.0 301.0
## Chrysler Imperial 14.7 440.0
## Duster 360        14.3 360.0
## Camaro Z28        13.3 350.0
## Cadillac Fleetwood 10.4 472.0
## Lincoln Continental 10.4 460.0
```

QUESTION: Which car has the highest fuel efficiency? ANSWER: Toyota Corolla

QUESTION: What is the size of it's engine? ANSWER: 71.1

TASK: Use code to create a plot comparing disp vs hp from the 'mtcars' dataset. With the following: Use the 'disp' data for the x-axis and labeled 'Engine Size (cubic inches)' Use the 'hp' data for the y-axis and labeled 'Horsepower (HP)' Add an appropriate title for the plot with a 'bold' font Add a legend colored by 'car_name' and labeled "Car Types"



Lets organize the horsepower from highest to lowest. Run the following.

```
mtcars %>%
  select(hp,disp) %>%
  arrange(desc(hp))
```

##		hp	disp
##	Maserati Bora	335	301.0
##	Ford Pantera L	264	351.0
##	Duster 360	245	360.0
##	Camaro Z28	245	350.0
##	Chrysler Imperial	230	440.0
##	Lincoln Continental	215	460.0
##	Cadillac Fleetwood	205	472.0
##	Merc 450SE	180	275.8
##	Merc 450SL	180	275.8
##	Merc 450SLC	180	275.8
##	Hornet Sportabout	175	360.0
##	Pontiac Firebird	175	400.0
##	Ferrari Dino	175	145.0
##	Dodge Challenger	150	318.0
##	AMC Javelin	150	304.0
##	Merc 280	123	167.6
##	Merc 280C	123	167.6
##	Lotus Europa	113	95.1
##	Mazda RX4	110	160.0
##	Mazda RX4 Wag	110	160.0
##	Hornet 4 Drive	110	258.0
##	Volvo 142E	109	121.0
##	Valiant	105	225.0
##	Toyota Corona	97	120.1
##	Merc 230	95	140.8
##	Datsun 710	93	108.0
##	Porsche 914-2	91	120.3
##	Fiat 128	66	78.7
##	Fiat X1-9	66	79.0
##	Toyota Corolla	65	71.1
##	Merc 240D	62	146.7
##	Honda Civic	52	75.7

QUESTION: What trend is the plot showing between engine size and horsepower? ANSWER: cars with smaller engine have lower hp

QUESTION: Which cars have the highest and lowest hp? ANSWER: highest is the Maserati Bora, lowest is the Honda civic

Don't forget to save and commit your changes.

Section 4: Add Visualizations and Tables to Your Report

Author: Themesha Parker (tlparker@uncg.edu)

Goal

In this section, you will create graphs and a summary table using the mtcars data set. You will also practice describing patterns that you observe in the output.

Before beginning, make sure you can access the mtcars dataset. Run the line below to preview the first few rows.

```
head(mtcars)
```

Also make sure you have Tidyverse installed! Run the line of code below.

```
library(tidyverse)
```

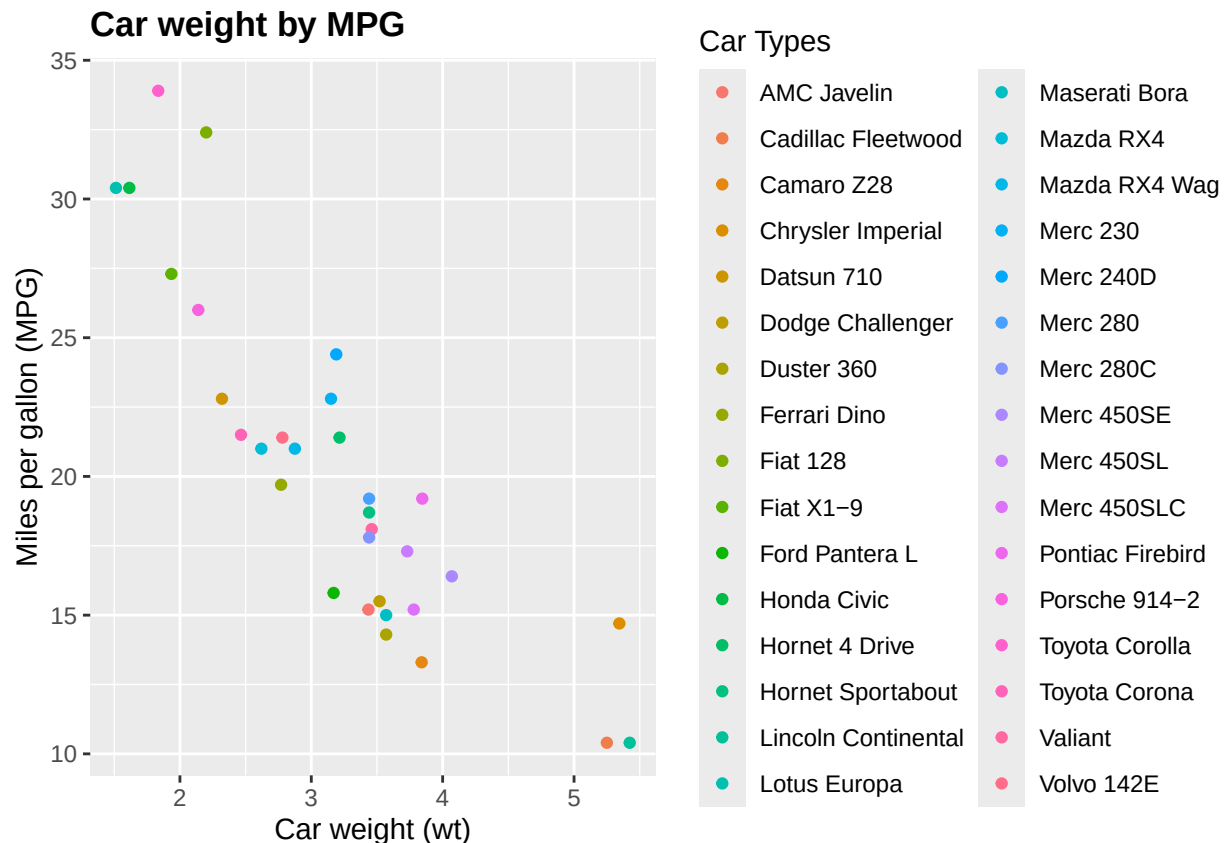
Part A: Create a Scatterplot

The variables `wt` and `mpg` in the `mtcars` data set can help us explore the relationship between car weight and fuel efficiency.

TASK:

Create a scatter plot with `wt` on the x-axis and `mpg` on the y-axis.

Hint: use the `plot()` function



TASK:

Modify your scatterplot so that it includes: a title, an x-axis label, a y-axis label, a point color of your choice

Hint: Think back to previous assignments(`main=`, `xlab=`, `ylab=`, and `col=`)

```
# Write your updated scatterplot code below this line
```

```
# this could just be the same thing I did above??
```

QUESTION: What does each point in this scatterplot represent? ANSWER: each individual car

QUESTION: What relationship do you observe between weight and mpg? ANSWER: lighter cars have higher mpg

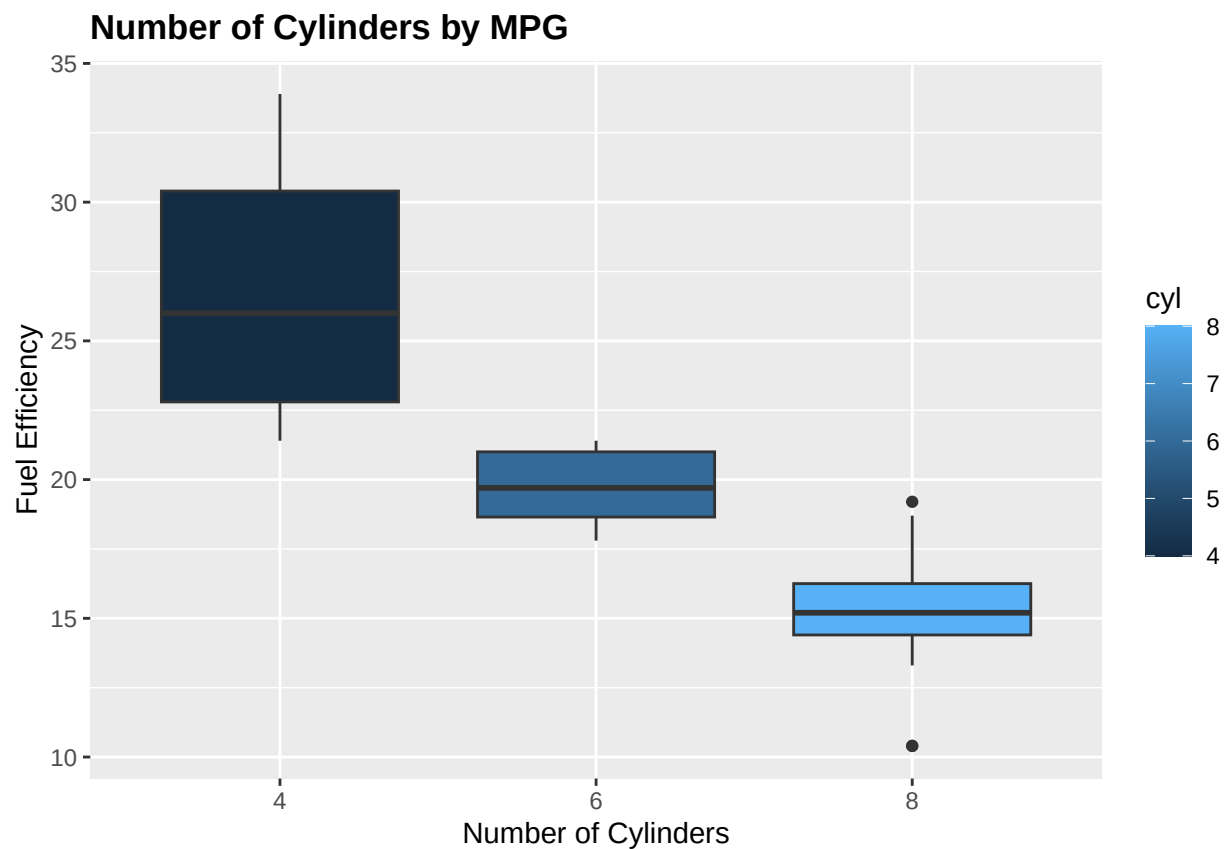
QUESTION: Does the graph suggest a positive relationship, a negative relationship, or no relationship? Explain your answer. ANSWER: this would be a negative relationship, because the points trend downward from left to right

Part B: Create a Boxplot

The variable `cyl` shows the number of cylinders in each car. A boxplot can help compare fuel efficiency across cylinder groups.

TASK:

Create a boxplot of `mpg` by `cyl` using the `boxplot` function



TASK:

Modify your boxplot so that it includes: a title, an x-axis label, a y-axis label, and a fill color of your choice

```
# Write your updated boxplot code below this line
```

```
# again, already did this in the code above
```

QUESTION: What does each box in this plot represent? ANSWER: the number of cylinders each car has and the MPG

QUESTION: Which cylinder group appears to have the highest fuel efficiency? ANSWER: cars with 4 cylinders

QUESTION: Which cylinder group appears to have the lowest fuel efficiency? ANSWER: cars with 2 cylinders

QUESTION: Based on the boxplot, how does fuel efficiency seem to change as the number of cylinders increases? ANSWER: it gets lower

Part C: Create a Summary Table

You will now create a table showing the average miles per gallon for each cylinder group.

TASK:

Use dplyr to calculate the average mpg for each value of cyl. Rename the data avg_mpg_by_cyl

Hint: you need to group the data by cyl then summarize the mean of mpg.

Use the space below to write your code:

```
# Write your code below this line
avg_mpg_by_cyl <- mtcars %>%
  group_by(cyl) %>%
  summarize(avg_mpg = mean(mpg))
```

TASK:

Display your results as a clean table.

Use the space below to write your code:

```
# Write your code below this line
print(avg_mpg_by_cyl)
```

```
## # A tibble: 3 x 2
##   cyl avg_mpg
##   <dbl> <dbl>
## 1     4    26.7
## 2     6    19.7
## 3     8    15.1
```

QUESTION: What does this table show? ANSWER: average mpg for each number of cylinders within the mtcars dataset

QUESTION: Which cylinder group has the highest average mpg? ANSWER: 4 cyl

QUESTION: Which cylinder group has the lowest average mpg? ANSWER: 8 cyl

Part D: Compare the Visuals

Now that you have created both graphs and the table, compare the information they provide.

QUESTION: How does the scatterplot help you understand the relationship between weight and fuel efficiency? ANSWER: we can see a trendline

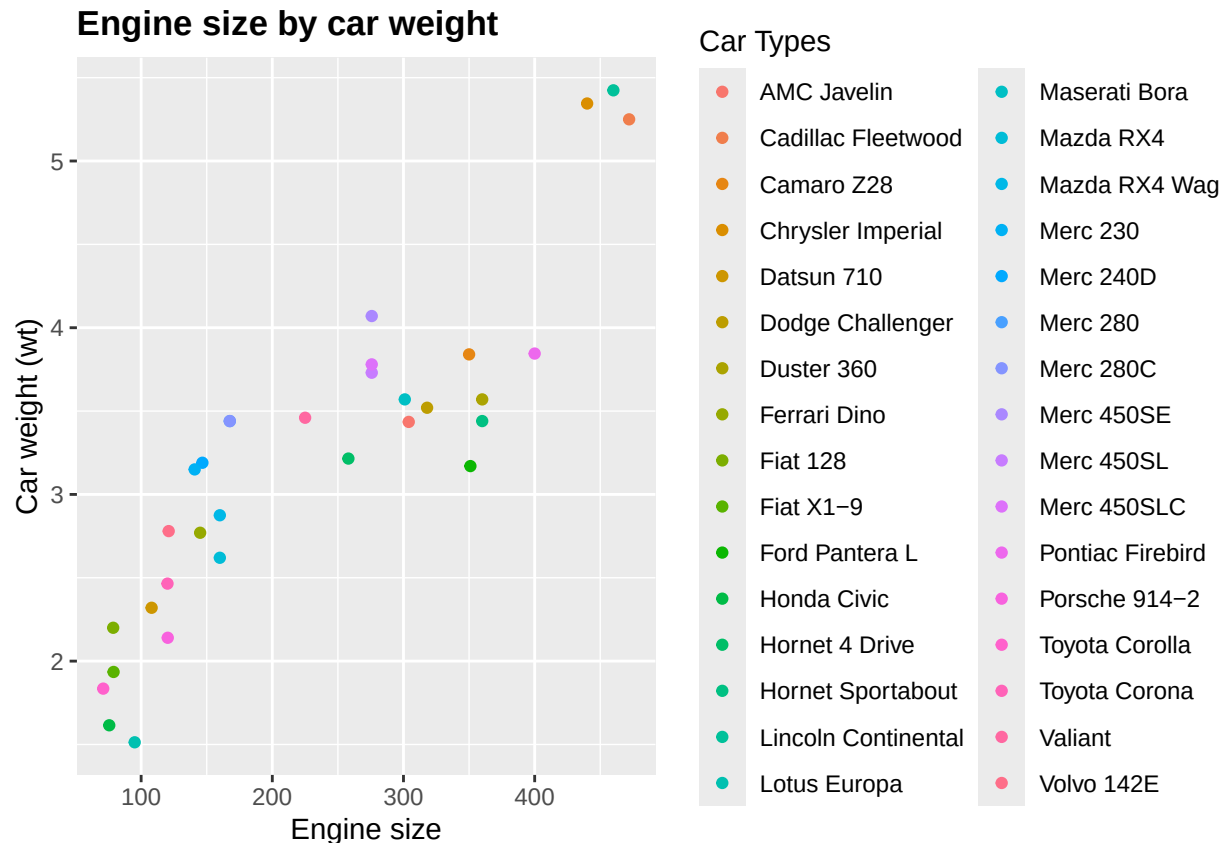
QUESTION: How does the boxplot help you compare different groups of cars? ANSWER: we group by a certain variable (cylinders in this case) to see the range within each group

Part E: Now you try

1. Use a different variable from mtcars to make an additional scatter plot.
2. Change the color of your scatterplot.

Write your code below:

```
# Write your code below this line
ggplot(mtcars, aes(x=disp, y=wt, color=factor(car_name))) +
  geom_point() +
  labs(
    x="Engine size",
    y="Car weight (wt)",
    title="Engine size by car weight",
    color="Car Types") +
  theme(plot.title = element_text(face="bold"))
```



QUESTION: What did you change, and how did it affect the output? ANSWER: when changing my x variable to disp and y variable to car weight, I now see a positive trend line in the scatterplot

Section 5: Improve and Finalize Your R Markdown Report

Objective: Learn how to make your report more polished, readable, and professional. In this section, we will take everything we have learned and improve the overall quality of our R Markdown report so it is easier to read and more useful.

Author: Dhanush Devanand (s_devanand@uncg.edu)

Step 1: Understanding Inline R Code

So far, we have been writing code in chunks. However, sometimes we want to include results directly inside our text. **Inline R code** allows us to insert calculations into sentences without a separate code chunk.

TASK:

Copy and paste the following lines directly into your R Markdown document, **outside** of any code chunk, and then Knit:

```
The mtcars dataset has 32 rows and 12 columns.  
The average miles per gallon (mpg) is 20.09.
```

Check

Did the numbers appear automatically in your rendered sentence?

QUESTION: What is inline R code, and how is it different from a regular code chunk? ANSWER: It pulls from the dataset to get the variables, but it is not being used to actually write code that would build a graph

QUESTION: Why is inline R code useful when working with a dataset that might be updated or changed over time? ANSWER: it will update accordingly when the dataset changes

TASK:

Now try writing your own inline R code sentence. Using the mtcars dataset, write a sentence that includes the maximum horsepower value using inline R code. The function `max()` will be helpful here.

Your sentence:

```
The mtcars dataset has 32 rows and 12 columns.  
The maximum horsepower value is '335.'
```

Why This Matters

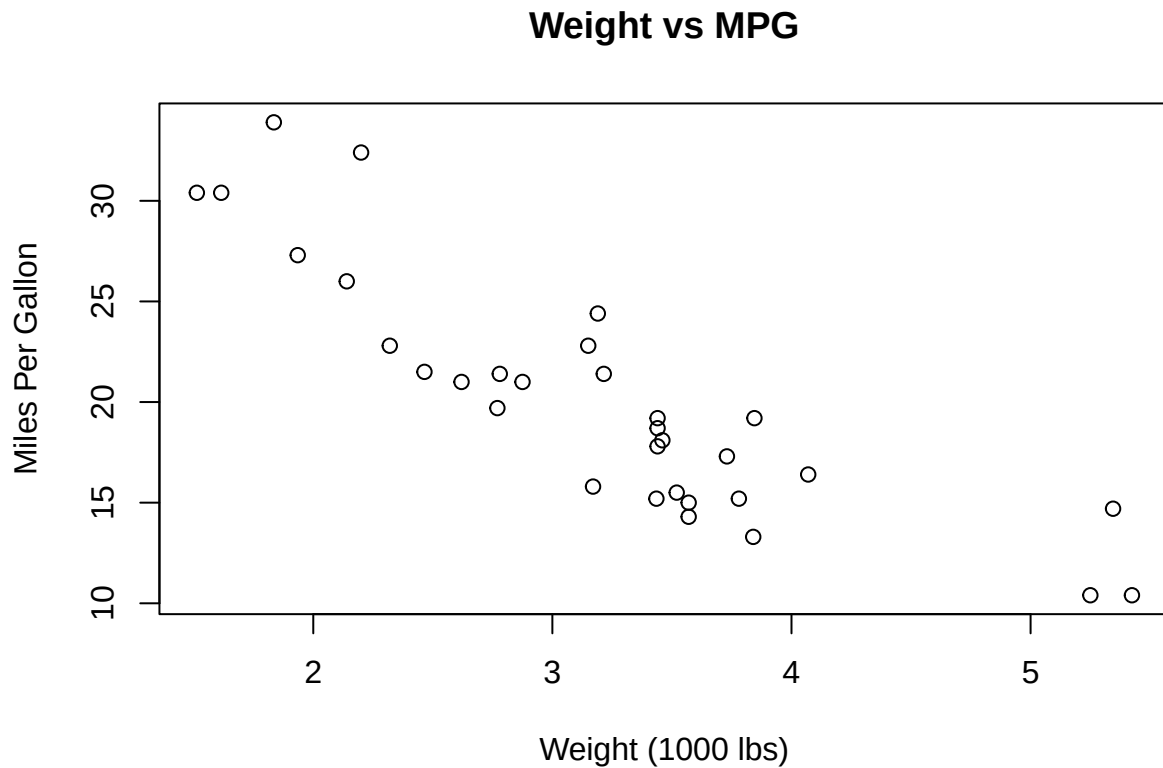
Inline R code makes your report dynamic. If the dataset changes, your results will automatically update without you needing to rewrite anything. This is especially important in professional or scientific reports where accuracy matters.

Step 2: Improving Readability by Hiding Code

Sometimes, showing all code can make a report look cluttered. If we only care about the output — like a graph — we can hide the code using the `echo=FALSE` chunk option.

TASK:

The chunk below currently shows the code when knitted. Add `echo=FALSE` inside the curly braces so that only the plot appears in the output, not the code. Then Knit your document.



Hint: Your chunk header should look like this when you are done: ````{r echo=FALSE}`

Check

Do you see the plot in the output but **not** the code that created it?

QUESTION: Why might you want to hide code in a final report? ANSWER: the code itself might not be important to the viewer

QUESTION: Can you think of a situation where you would want to **show** your code in a report rather than hide it? Who might be reading that report? ANSWER: if you are showing it to someone who wants to replicate your process, they will need to see how you wrote it!

TASK:

Now apply `echo=FALSE` to the ggplot chunk you created back in Section 3. Knit the document and confirm only the plot appears.

Step 3: Removing Warnings and Messages

When you load packages or run certain functions, R sometimes prints extra warnings and messages that are not important to the reader. These can make your report look messy. You can suppress them using the chunk options `message=FALSE` and `warning=FALSE`.

TASK: — Part A: See the messages first

Run the chunk below **without** any chunk options and Knit. Read through the output carefully and write down any extra messages or warnings you notice below.

```
library(tidyverse)
summary(mtcars)
```

```
##           mpg           cyl           disp           hp
##  Min.      :10.40   Min.       :4.000   Min.       : 71.1   Min.       : 52.0
## 1st Qu.:15.43   1st Qu.:4.000   1st Qu.:120.8   1st Qu.: 96.5
## Median :19.20   Median :6.000   Median :196.3   Median :123.0
## Mean   :20.09   Mean   :6.188   Mean   :230.7   Mean   :146.7
## 3rd Qu.:22.80   3rd Qu.:8.000   3rd Qu.:326.0   3rd Qu.:180.0
## Max.    :33.90   Max.    :8.000   Max.    :472.0   Max.    :335.0
##           drat           wt           qsec           vs
##  Min.      :2.760   Min.      :1.513   Min.      :14.50   Min.      :0.0000
## 1st Qu.:3.080   1st Qu.:2.581   1st Qu.:16.89   1st Qu.:0.0000
## Median :3.695   Median :3.325   Median :17.71   Median :0.0000
## Mean   :3.597   Mean   :3.217   Mean   :17.85   Mean   :0.4375
## 3rd Qu.:3.920   3rd Qu.:3.610   3rd Qu.:18.90   3rd Qu.:1.0000
## Max.    :4.930   Max.    :5.424   Max.    :22.90   Max.    :1.0000
##           am           gear           carb           car_name
##  Min.      :0.0000   Min.      :3.000   Min.      :1.000   Length:32
## 1st Qu.:0.0000   1st Qu.:3.000   1st Qu.:2.000   Class :character
## Median :0.0000   Median :4.000   Median :2.000   Mode  :character
## Mean   :0.4062   Mean   :3.688   Mean   :2.812
## 3rd Qu.:1.0000   3rd Qu.:4.000   3rd Qu.:4.000
## Max.    :1.0000   Max.    :5.000   Max.    :8.000
```

What messages or warnings did you see? I am actually not seeing any warning messages...

TASK: — Part B: Suppress the messages

Now add `message=FALSE` and `warning=FALSE` to the chunk header above and Knit again.

Hint: Your chunk header should look like this when you are done: ``{r message=FALSE, warning=FALSE}`

Check

Did the extra messages disappear from your rendered output?

QUESTION: What is the difference between `message=FALSE` and `warning=FALSE`? When might you need to use both at the same time? ANSWER: `message=FALSE` only hides messages, and `warning=FALSE` hides warnings. You may use both at the same time if you know that neither warnings or messages are relevant to your output

QUESTION: Is it always a good idea to suppress warnings? Can you think of a case where seeing a warning might actually be important? ANSWER: sometimes warnings are telling you that some of your data is not being shown, which you would probably want to fix before moving forward

Why This Matters

Hiding unnecessary messages keeps your report clean and professional, especially when sharing results with someone who does not need to see behind-the-scenes R output. However, always make sure you understand what a warning means before choosing to hide it.

Step 4: Hiding Code AND Output with `include=FALSE`

In Step 2, you learned that `echo=FALSE` hides the code but still shows the output. Sometimes you may want to run a chunk silently — hiding both the code and the output entirely. This is done with `include=FALSE`.

This is useful for setup steps like loading libraries or defining variables that need to run but do not need to appear in the report.

TASK:

Run the chunk below as-is and Knit. Note what appears in the output. Then change the chunk option from `echo=FALSE` to `include=FALSE` and Knit again.

```
##   mean_mpg max_hp min_wt
## 1    20.09   335  1.513
```

Hint: Your chunk header should look like this when you are done: ````{r include=FALSE}`

Check

When using `include=FALSE`, does anything from this chunk appear in the rendered output?

QUESTION: What is the difference between `echo=FALSE` and `include=FALSE`? ANSWER: `echo=FALSE` hides the code but shows the output. `include=FALSE` hides everything

QUESTION: Give an example of when you would use `include=FALSE` in a real report. ANSWER: as mentioned above, we may want to hide the code used to instal/load packages, because that just takes up space in a document

TASK:

Now use the `my_summary` object created in the chunk above inside an inline R code sentence below. Even though the chunk is hidden, the object is still available to use. Copy and paste the lines below **outside** of a code chunk and Knit to confirm the values appear.

The average miles per gallon across all cars is 20.09 and the most powerful engine produces 335 horsepower

Check

Did the inline values populate correctly even though the chunk that created them was hidden? Yes!

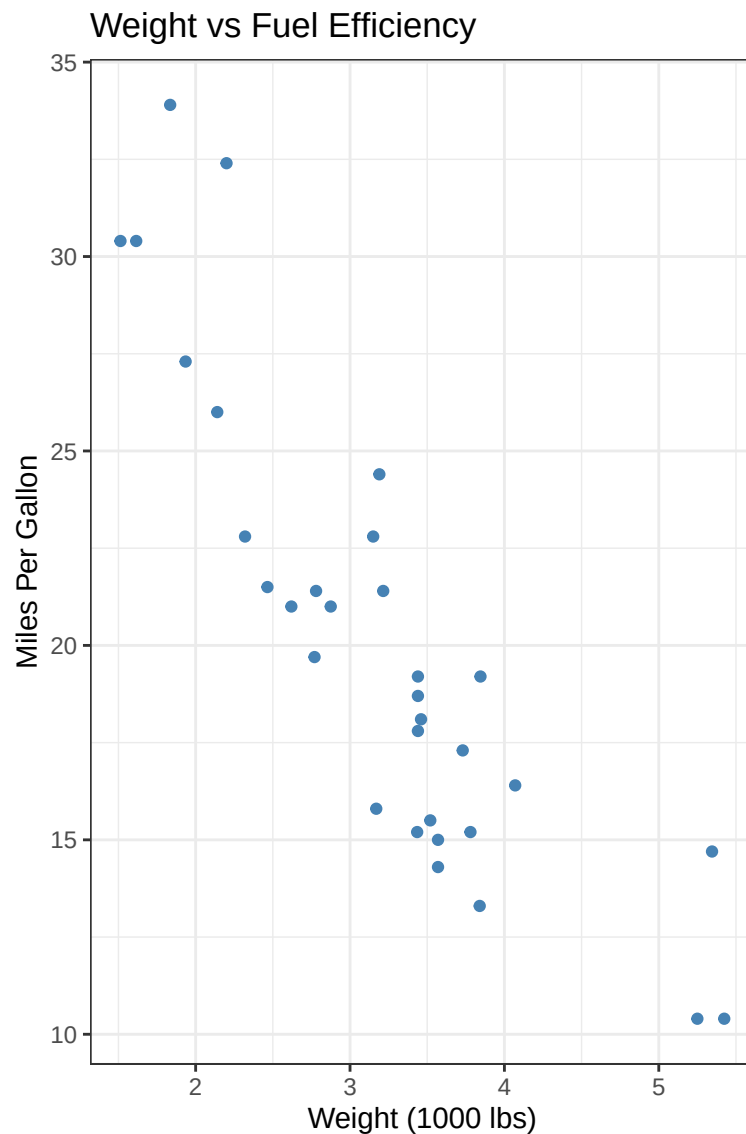
Step 5: Controlling Figure Size

When creating plots in R Markdown, you can control the size of your figures directly in the chunk header using `fig.width` and `fig.height`. This helps make your report look clean and consistent.

TASK:

Run the chunk below and Knit to see the default figure size.

```
ggplot(mtcars, aes(x = wt, y = mpg)) +  
  geom_point(color = "steelblue") +  
  labs(title = "Weight vs Fuel Efficiency",  
        x = "Weight (1000 lbs)",  
        y = "Miles Per Gallon") +  
  theme_bw()
```



TASK:

Now add `fig.width=8` and `fig.height=4` to the chunk header and Knit again.

Hint: Your chunk header should look like this when you are done: ````{r fig.width=8, fig.height=4}`

Check

Did the shape of the plot change in your rendered output?

QUESTION: How did changing `fig.width` and `fig.height` affect the appearance of your plot? ANSWER: it made it fit the width of the screen

QUESTION: Why might consistent figure sizing matter in a professional or academic report? ANSWER: It allows us to follow the formatting guidelines provided by different publications

TASK:

Try setting `fig.width=4` and `fig.height=6` on the same chunk. Knit and observe how the plot changes. Describe what you notice below.

Observation: It is now stretched longer, taking up more space and kind of damaging the quality of the graph

Step 6: Adding a Table of Contents

A table of contents makes it easier to navigate your report, especially as it gets longer. You can add one by modifying the YAML header at the very top of your `.Rmd` file.

TASK:

Make sure the YAML header at the very top of your document looks like this, then Knit:

```
---
title: "Group5_RMarkdown"
author: "Arnav Pandey, Themesha Parker, Michelle Bautista Canuto, Dhanush Devanand"
date: "2026-04-26"
output:
  html_document:
    toc: true
---
```

Check

Do you now see a table of contents at the top of your rendered HTML document?

QUESTION: How does a table of contents improve the readability of a long report? ANSWER: it helps users move through your document and skip to relevant parts

QUESTION: The table of contents is automatically generated from your headings. What does this tell you about the importance of using proper heading levels (`#`, `##`, `###`) throughout your document? ANSWER: correct heading is necessary for the table of contents to work

Step 7: Adding Hyperlinks

You can include clickable links in your report to point readers to additional resources. The syntax is `[link text](URL)`.

TASK:

Add the following clickable link somewhere in your document and Knit:

[R Markdown documentation](https://rmarkdown.rstudio.com)

Check

Does the link appear clickable after knitting? yes

TASK:

Now add a second hyperlink of your own. Find a webpage related to something covered in this assignment — for example, the `mtcars` dataset, `ggplot2`, or R Markdown chunk options — and add it below using the same syntax.

Your hyperlink: [Data visualization cheatsheet](https://rstudio.github.io/cheatsheets/html/data-visualization.html)

QUESTION: When might adding a hyperlink be useful in a data report? ANSWER: to help explain certain aspects of your code without typing it all out in the script itself

Step 8: Writing a Strong Conclusion

Every report should end with a short conclusion that summarizes what was done and what was found.

TASK:

Write a short paragraph (3–5 sentences) below summarizing what you did in this assignment. Consider:

- What dataset did you use?
- What visualizations or analyses did you create?
- What improvements did you make to the report in Section 5?
- Why is R Markdown useful for communicating data?

Your conclusion: This assignment worked with the `mtcars` dataset in order to build various scatterplots and barplots. These plots helped us to visualize relationships between different variables within each car, such as the influence of number of cylinders within a car on overall horsepower or MPG. In section 5, we learned how to improve the document by writing inline code and hiding specific portions of the text/code. R Markdown is useful because it allows us to transfer our code into different formats, which oftentimes are more familiar to a general audience (looking at script in an R environment can be kind of intimidating at first).

TASK:

Now rewrite at least one sentence from your conclusion so that it includes inline R code. For example, instead of writing “the dataset has 32 rows,” use `32` to generate that number automatically.

Your updated sentence with inline R code:

``The mtcars dataset has 32 rows and 12 columns.``

Step 9: Final Check and Knit

Before submitting, go through the checklist below to make sure everything is working correctly. In your `.Rmd` file, replace each `[ ]` with `[x]` once you have confirmed that step.

- ☒ Inline R code displays correctly in rendered text
- ☒ Plots appear in the output without showing code (`echo=FALSE`)
- ☒ A chunk runs silently using `include=FALSE`
- ☒ No unnecessary warnings or messages appear (`message=FALSE`, `warning=FALSE`)
- ☒ Figure sizes have been adjusted using `fig.width` and `fig.height`
- ☒ Table of contents is visible at the top of the document
- ☒ At least two hyperlinks are present and clickable
- ☒ Conclusion paragraph is included with at least one inline R code value

Once all boxes are checked, click **Knit** one final time to generate your completed report.